

# 解题能力训练-DP

清华大学计算机科学与技术系 顾霆枫

# DP 问题一页纸

- ▶ Step1 划分阶段，寻找决策，确定状态
  - ▶ 将影响后续决策的因素计入状态
  - ▶ 优化掉不影响后续决策的因素
- ▶ Step2 明确转移，写出转移关系
  - ▶ 对每个状态，写出每种决策使其转移到哪种状态
  - ▶ 写出决策对目标值的影响
- ▶ Step3 倒推状态转移方程
  - ▶ 把转移箭头右侧的下标统一
  - ▶ 合并转移式
- ▶ Step4 确定DP顺序，**确定边界情况，写出朴素DP**
- ▶ (Step5) 添加优化

# 课程计划

上午讲历年联赛的一些DP题

下午讲上午没讲完的部分，再补充一些有趣的题目

# 简单题

# NOIP2007 矩阵取数游戏

帅帅经常跟同学玩一个矩阵取数游戏：对于一个给定的  $n \times m$  的矩阵，矩阵中的每个元素  $a_{i,j}$  均为非负整数。游戏规则如下：

1. 每次取数时须从每行各取走一个元素，共  $n$  个。经过  $m$  次后取完矩阵内所有元素；
2. 每次取走的各个元素只能是该元素所在行的行首或行尾；
3. 每次取数都有一个得分值，为每行取数的得分之和，每行取数的得分 = 被取走的元素值  $\times 2^i$ ，其中  $i$  表示第  $i$  次取数（从 1 开始编号）；
4. 游戏结束总得分为  $m$  次取数得分之和。

帅帅想请你帮忙写一个程序，对于任意矩阵，可以求出取数后的最大得分。

数据范围：

60% 的数据满足：  $1 \leq n, m \leq 30$ ，答案不超过  $10^{16}$ 。

100% 的数据满足：  $1 \leq n, m \leq 80$ ，  $0 \leq a_{i,j} \leq 1000$ 。

# NOIP2007 矩阵取数游戏

- ▶ 首先，行与行之间决策和状态完全独立，可以对每行分别求解最后相加

# NOIP2007 矩阵取数游戏

- ▶ 首先，行与行之间决策和状态完全独立，可以对每行分别求解最后相加
- ▶ 注意到当一行取得只剩 $i \sim j$ 时，下一步的权重是确定的，且决策和之前的选择无关

# NOIP2007 矩阵取数游戏

- ▶ 首先，行与行之间决策和状态完全独立，可以对每行分别求解最后相加
- ▶ 注意到当一行取得只剩 $i \sim j$ 时，下一步的权重是确定的，且决策和之前的选择无关
- ▶ 很容易想到区间DP
- ▶  $f[i][j]$ 表示一行取得只剩 $i \sim j$ 时的最大得分
- ▶  $f[i][j] + 2^{(m-j+i)} a[i] \rightarrow f[i+1][j]$
- ▶  $f[i][j] + 2^{(m-j+i)} a[j] \rightarrow f[i][j-1]$
- ▶  $f[i][j] = \max\{f[i-1][j] + 2^{(m-j+i+1)} a[i-1], f[i][j+1] + 2^{(m-j+i-1)} a[j+1]\}$

# NOIP2010 乌龟棋

## 题目描述

乌龟棋的棋盘是一行  $N$  个格子，每个格子上有一个分数（非负整数）。棋盘第1格是唯一的起点，第  $N$  格是终点，游戏要求玩家控制一个乌龟棋子从起点出发走到终点。

乌龟棋中  $M$  张爬行卡片，分成4种不同的类型（ $M$  张卡片中不一定包含所有4种类型的卡片，见样例），每种类型的卡片上分别标有1, 2, 3, 4四个数字之一，表示使用这种卡片后，乌龟棋子将向前爬行相应的格子数。游戏中，玩家每次需要从所有的爬行卡片中选择一张之前没有使用过的爬行卡片，控制乌龟棋子前进相应的格子数，每张卡片只能使用一次。

游戏中，乌龟棋子自动获得起点格子的分数，并且在后续的爬行中每到达一个格子，就得到该格子相应的分数。玩家最终游戏得分就是乌龟棋子从起点到终点过程中到过的所有格子的分数总和。

很明显，用不同的爬行卡片使用顺序会使得最终游戏的得分不同，小明想要找到一种卡片使用顺序使得最终游戏得分最多。

现在，告诉你棋盘上每个格子的分数和所有的爬行卡片，你能告诉小明，他最多能得到多少分吗？

对于100%的数据有  $1 \leq N \leq 350, 1 \leq M \leq 120$ ，且4种爬行卡片，每种卡片的张数不会超过40；  
 $0 \leq a_i \leq 100, 1 \leq i \leq N, 1 \leq b_i \leq 4, 1 \leq i \leq M$ 。

# NOIP2010 乌龟棋

- ▶ 题目描述不难理解

# NOIP2010 乌龟棋

- ▶ 题目描述不难理解
- ▶ 分析题目给出的条件，决策是要使用哪一张爬行卡
- ▶ 当前所处的位置，每种爬行卡使用过的张数都会对之后的决策产生影响
- ▶ 同时，只要确定了各个爬行卡使用过的张数，之后的决策就不受之前爬行卡使用顺序的影响

# NOIP2010 乌龟棋

- ▶ 题目描述不难理解
- ▶ 分析题目给出的条件，决策是要使用哪一张爬行卡
- ▶ 当前所处的位置，每种爬行卡使用过的张数都会对之后的决策产生影响
- ▶ 同时，只要确定了各个爬行卡使用过的张数，之后的决策就不受之前爬行卡使用顺序的影响
- ▶ 用  $f[i][j][k][l]$  表示4种爬行卡分别使用  $i, j, k, l$  张后的最大得分
- ▶ 方程和转移都很简单

# NOIP2018 货币系统

## 题目描述

收起

在网友的国度中共有  $n$  种不同面额的货币，第  $i$  种货币的面额为  $a[i]$ ，你可以假设每一种货币都有无穷多张。为了方便，我们把货币种数为  $n$ 、面额数组为  $a[1..n]$  的货币系统记作  $(n, a)$ 。

在一个完善的货币系统中，每一个非负整数的金额  $x$  都应该可以被表示出，即对每一个非负整数  $x$ ，都存在  $n$  个非负整数  $t[i]$  满足  $a[i] \times t[i]$  的和为  $x$ 。然而，在网友的国度中，货币系统可能是不完善的，即可能存在金额  $x$  不能被该货币系统表示出。例如在货币系统  $n = 3, a = [2, 5, 9]$  中，金额  $1, 3$  就无法被表示出来。

两个货币系统  $(n, a)$  和  $(m, b)$  是等价的，当且仅当对于任意非负整数  $x$ ，它要么均可以被两个货币系统表出，要么不能被其中任何一个表出。

现在网友们打算简化一下货币系统。他们希望找到一个货币系统  $(m, b)$ ，满足  $(m, b)$  与原来的货币系统  $(n, a)$  等价，且  $m$  尽可能的小。他们希望你来协助完成这个艰巨的任务：找到最小的  $m$ 。

【数据范围与约定】

| 测试点 | $n$ | $a_i$       | 测试点 | $n$        | $a_i$        |
|-----|-----|-------------|-----|------------|--------------|
| 1   | = 2 | $\leq 1000$ | 11  | $\leq 13$  | $\leq 16$    |
| 2   |     |             | 12  |            |              |
| 3   |     |             | 13  |            |              |
| 4   | = 3 |             | 14  | $\leq 25$  | $\leq 40$    |
| 5   |     |             | 15  |            |              |
| 6   |     |             | 16  |            |              |
| 7   | = 4 |             | 17  | $\leq 100$ | $\leq 25000$ |
| 8   |     |             | 18  |            |              |
| 9   | = 5 |             | 19  |            |              |
| 10  |     |             | 20  |            |              |

对于 100% 的数据，满足  $1 \leq T \leq 20, n, a[i] \geq 1$ 。

# NOIP2018 货币系统

- ▶ 先给出一个结论，我们所需要的 $b$ 一定是 $a$ 的子集，且某个面值在 $b$ 中当且仅当它不能由 $a$ 中的其他面值组合而成。
- ▶ 这个结论十分符合直觉，若猜到，也不难证明。
- ▶ 于是问题转化为找到 $a$ 中所有不能被其它面值的币种组和表出的币种。

# NOIP2018 货币系统

- ▶ 先给出一个结论，我们所需要的 $b$ 一定是 $a$ 的子集，且某个面值在 $b$ 中当且仅当它不能由 $a$ 中的其他面值组合而成。
- ▶ 这个结论十分符合直觉，若猜到，也不难证明。
- ▶ 于是问题转化为找到 $a$ 中所有不能被其它面值的币种组和表出的币种。
- ▶ 我们可以用 $f[x]=0/1$ 来记录 $x$ 能否已处理的面值被表出，初始 $f[0]=1$ ，其余为0
- ▶ 从小到大处理每一个 $a$ 中的面值
- ▶ 如果 $f[a_i]=0$ ，说明 $a_i$ 不能被面值小于它的货币表出，将 $a_i$ 加入 $b$
- ▶ 之后从小到大扫描 $f$ 中的每一个位置，若 $f[x]=1$ ，则 $f[x+a_i]=1$ （处理到 $\max a$ 即可）
- ▶ 扫遍 $a$ 后得到的 $b$ 即为答案

# 中等题

# NOIP2006 能量项链

## 题目描述

$N(4 \leq N \leq 100)$

[展开](#)

在 *Mars* 星球上，每个 *Mars* 人都随身佩带着一串能量项链。在项链上有  $N$  颗能量珠。能量珠是一颗有头标记与尾标记的珠子，这些标记对应着某个正整数。并且，对于相邻的两颗珠子，前一颗珠子的尾标记一定等于后一颗珠子的头标记。因为只有这样，通过吸盘（吸盘是 *Mars* 人吸收能量的一种器官）的作用，这两颗珠子才能聚合成一颗珠子，同时释放出可以被吸盘吸收的能量。如果前一颗能量珠的头标记为  $m$ ，尾标记为  $r$ ，后一颗能量珠的头标记为  $r$ ，尾标记为  $n$ ，则聚合后释放的能量为  $m \times r \times n$  (*Mars* 单位)，新产生的珠子的头标记为  $m$ ，尾标记为  $n$ 。

需要时，*Mars* 人就用吸盘夹住相邻的两颗珠子，通过聚合得到能量，直到项链上只剩下一颗珠子为止。显然，不同的聚合顺序得到的总能量是不同的，请你设计一个聚合顺序，使一串项链释放出的总能量最大。

例如：设  $N = 4$ ，4 颗珠子的头标记与尾标记依次为  $(2, 3)(3, 5)(5, 10)(10, 2)$ 。我们用记号  $\oplus$  表示两颗珠子的聚合操作， $(j \oplus k)$  表示第  $j, k$  两颗珠子聚合后所释放的能量。则第 4、1 两颗珠子聚合后释放的能量为：

$$(4 \oplus 1) = 10 \times 2 \times 3 = 60。$$

这一串项链可以得到最优值的一个聚合顺序所释放的总能量为：

$$((4 \oplus 1) \oplus 2) \oplus 3 = 10 \times 2 \times 3 + 10 \times 3 \times 5 + 10 \times 5 \times 10 = 710。$$

# NOIP2006 能量项链

- ▶ 一道经典的环形区间DP

# NOIP2006 能量项链

- ▶ 一道经典的环形区间DP
- ▶ 对于非环形的区间DP，常规做法是 $dp[l][r]$ 计算 $l \sim r$ 这一段合并的最大价值
- ▶ 在枚举分割点，转移 $O(n)$ 的情况下，这一做法拥有 $O(n^3)$ 的时间复杂度

# NOIP2006 能量项链

- ▶ 一道经典的环形区间DP
- ▶ 对于非环形的区间DP，常规做法是 $dp[l][r]$ 计算 $l \sim r$ 这一段合并的最大价值
- ▶ 在枚举分割点，转移 $O(n)$ 的情况下，这一做法拥有 $O(n^3)$ 的时间复杂度
- ▶ 在环形的情况下，一个不优秀的做法是，枚举一个分割点，把环分割成一条链做区间DP，使得复杂度生生来到  $O(n^4)$

# NOIP2006 能量项链

- ▶ 一道经典的环形区间DP
- ▶ 对于非环形的区间DP，常规做法是 $dp[l][r]$ 计算 $l \sim r$ 这一段合并的最大价值
- ▶ 在枚举分割点，转移 $O(n)$ 的情况下，这一做法拥有 $O(n^3)$ 的时间复杂度
- ▶ 在环形的情况下，一个不优秀的做法是，枚举一个分割点，把环分割成一条链做区间DP，使得复杂度生生来到  $O(n^4)$
- ▶ 实际上，对于环形区间DP，常用的状态是 $dp[i][len]$ 计算从 $i$ 往后推 $len$ 个位置的一段合并的最大价值
- ▶ DP顺序按 $len$ 从小往大即可

# NOIP2007 树网的核

## 题目描述

收起

设  $T = (V, E, W)$  是一个无圈且连通的无向图（也称为无根树），每条边到有正整数的权，我们称  $T$  为树网（`treenetwork`），其中  $V, E$  分别表示结点与边的集合， $W$  表示各边长度的集合，并设  $T$  有  $n$  个结点。

路径：树网中任何两结点  $a, b$  都存在唯一的一条简单路径，用  $d(a, b)$  表示以  $a, b$  为端点的路径的长度，它是该路径上各边长度之和。我们称  $d(a, b)$  为  $a, b$  两结点间的距离。

$D(v, P) = \min\{d(v, u)\}$ ,  $u$  为路径  $P$  上的结点。

树网的直径：树网中最长的路径成为树网的直径。对于给定的树网  $T$ ，直径不一定是唯一的，但可以证明：各直径的中点（不一定恰好是某个结点，可能在某条边的内部）是唯一的，我们称该点为树网的中心。

偏心距  $ECC(F)$ ：树网  $T$  中距路径  $F$  最远的结点到路径  $F$  的距离，即

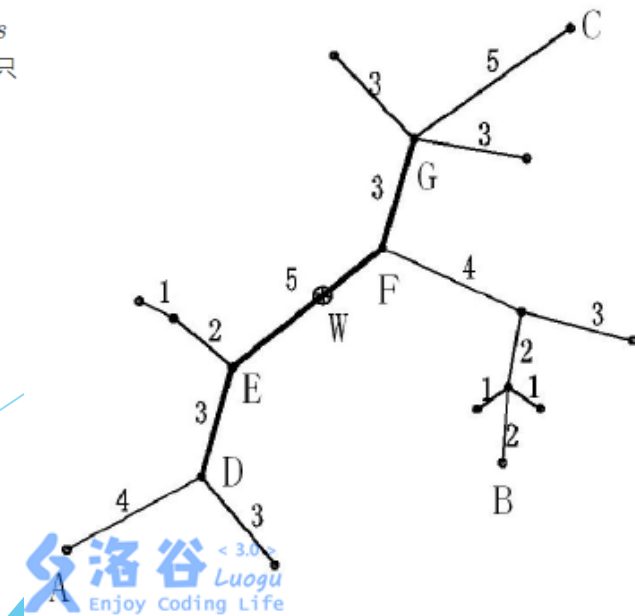
$$ECC(F) = \max\{d(v, F), v \in V\}$$

任务：对于给定的树网  $T = (V, E, W)$  和非负整数  $s$ ，求一个路径  $F$ ，他是某直径上的一段路径（该路径两端均为树网中的结点），其长度不超过  $s$ （可以等于  $s$ ），使偏心距  $ECC(F)$  最小。我们称这个路径为树网  $T = (V, E, W)$  的核（`Core`）。必要时， $F$  可以退化为某个结点。一般来说，在上述定义下，核不一定只有一个，但最小偏心距是唯一的。

下面的图给出了树网的一个实例。图中， $A - B$  与  $A - C$  是两条直径，长度均为 20。点  $W$  是树网的中心， $EF$  边的长度为 5。如果指定  $s = 11$ ，则树网的核为路径 `DEFG`（也可以取为路径 `DEF`），偏心距为 8。如果指定  $s = 0$ （或  $s = 1$ 、 $s = 2$ ），则树网的核为结点  $F$ ，偏心距为 12。

原题  $n \leq 300$ ，太水了

现在  $n \leq 10^6$



# NOIP2007 树网的核

- ▶ 题目废话太多，简化题意：
- ▶ 给出一棵带权无根树，求在直径上取一段长度不超过 $s$ 的路径，使得（树上任意一点到路径（上最近点）的距离）的最大值尽可能小

# NOIP2007 树网的核

- ▶ 题目废话太多，简化题意：
- ▶ 给出一棵带权无根树，求在直径上取一段长度不超过 $s$ 的路径，使得（树上任意一点到路径（上最近点）的距离）的最大值尽可能小
- ▶ 我们把问题拆开来解决

# NOIP2007 树网的核

- ▶ 题目废话太多，简化题意：
- ▶ 给出一棵带权无根树，求在直径上取一段长度不超过 $s$ 的路径，使得（树上任意一点到路径（上最近点）的距离）的最大值尽可能小
- ▶ 我们把问题拆开来解决
- ▶ 实际上，一条直径会把树分割成以直径上每个点为根的一棵棵子树，而这些子树中任意一点到根的距离就是它到直径的距离

# NOIP2007 树网的核

- ▶ 题目废话太多，简化题意：
- ▶ 给出一棵带权无根树，求在直径上取一段长度不超过 $s$ 的路径，使得（树上任意一点到路径（上最近点）的距离）的最大值尽可能小
- ▶ 我们把问题拆开来解决
- ▶ 实际上，一条直径会把树分割成以直径上每个点为根的一棵棵子树，而这些子树中任意一点到根的距离就是它到直径的距离
- ▶ 同时，直径满足这样一个性质，对于直径上两个点 $u, v$ ， $v$ 子树中的任意一点到 $u$ 的距离一定不大于 $u$ 到直径靠近 $v$ 这一侧的端点的距离
- ▶ （否则直径就可以被替换变长）

# NOIP2007 树网的核

- ▶ 那么，直径上每个点到它分得的子树的最长距离其实是可以预处理的，子树没有交集，所以预处理复杂度 $O(n)$

# NOIP2007 树网的核

- ▶ 那么，直径上每个点到它分得的子树的最长距离其实是可以预处理的，子树没有交集，所以预处理复杂度 $O(n)$
- ▶ 那么问题其实转化为，给直径上每一个点一个权值（等于到它分得的子树的最长距离），求直径上找长度不超过 $s$ 的一段，使得所选点权 $U$ 两侧未被选入的长度的最大值尽可能小

# NOIP2007 树网的核

- ▶ 那么，直径上每个点到它分得的子树的最长距离其实是可以预处理的，子树没有交集，所以预处理复杂度 $O(n)$
- ▶ 那么问题其实转化为，给直径上每一个点一个权值（等于到它分得的子树的最长距离），求直径上找长度不超过 $s$ 的一段，使得所选点权 $\cup$ 两侧未被选入的长度的最大值尽可能小
- ▶ 直觉告诉我们，核的长度选的尽可能接近 $s$ 越好（可以证明，还是从直径性质入手），而两侧未选入的长度可以 $O(1)$ 求出，我们只需要动态维护所选的点权的最大值

# NOIP2007 树网的核

- ▶ 那么，直径上每个点到它分得的子树的最长距离其实是可以预处理的，子树没有交集，所以预处理复杂度 $O(n)$
- ▶ 那么问题其实转化为，给直径上每一个点一个权值（等于到它分得的子树的最长距离），求直径上找长度不超过 $s$ 的一段，使得所选点权 $\cup$ 两侧未被选入的长度的最大值尽可能小
- ▶ 直觉告诉我们，核的长度选的尽可能接近 $s$ 越好（可以证明，还是从直径性质入手），而两侧未选入的长度可以 $O(1)$ 求出，我们只需要动态维护所选的点权的最大值
- ▶ 看起来越来越熟悉了——给定一个序列，队尾元素不断地加入，队首元素渐渐地失效，要维护队列的最大值

# NOIP2007 树网的核

- ▶ 那么，直径上每个点到它分得的子树的最长距离其实是可以预处理的，子树没有交集，所以预处理复杂度 $O(n)$
- ▶ 那么问题其实转化为，给直径上每一个点一个权值（等于到它分得的子树的最长距离），求直径上找长度不超过 $s$ 的一段，使得所选点权 $U$ 两侧未被选入的长度的最大值尽可能小
- ▶ 直觉告诉我们，核的长度选的尽可能接近 $s$ 越好（可以证明，还是从直径性质入手），而两侧未选入的长度可以 $O(1)$ 求出，我们只需要动态维护所选的点权的最大值
- ▶ 看起来越来越熟悉了——给定一个序列，队尾元素不断地加入，队首元素渐渐地失效，要维护队列的最大值
- ▶ 这显然是单调队列干的事情，保证队列内部单调递减即可

# NOIP2008 传纸条

## 题目描述

展开

小渊和小轩是好朋友也是同班同学，他们在一起总有谈不完的话题。一次素质拓展活动中，班上同学安排做成一个  $m$  行  $n$  列的矩阵，而小渊和小轩被安排在矩阵对角线的两端，因此，他们就无法直接交谈了。幸运的是，他们可以通过传纸条来进行交流。纸条要经由许多同学传到对方手里，小渊坐在矩阵的左上角，坐标  $(1, 1)$ ，小轩坐在矩阵的右下角，坐标  $(m, n)$ 。从小渊传到小轩的纸条只可以向下或者向右传递，从小轩传给小渊的纸条只可以向上或者向左传递。

在活动进行中，小渊希望给小轩传递一张纸条，同时希望小轩给他回复。班里每个同学都可以帮他们传递，但只会帮他们一次，也就是说如果此人在小渊递给小轩纸条的时候帮忙，那么在小轩递给小渊的时候就不会再帮忙。反之亦然。

还有一件事情需要注意，全班每个同学愿意帮忙的好感度有高有低（注意：小渊和小轩的好心程度没有定义，输入时用 0 表示），可以用一个  $[0, 100]$  内的自然数来表示，数越大表示越好心。小渊和小轩希望尽可能找好心程度高的同学来帮忙传纸条，即找到来回两条传递路径，使得这两条路径上同学的好心程度之和最大。现在，请你帮助小渊和小轩找到这样的两条路径。

# NOIP2008 传纸条

- ▶ 题意简化
  - ▶ 给出一个 $n*m$ 的矩阵，除 $(1,1)$ 和 $(n,m)$ 外每个格子中有一个分数
  - ▶ 从 $(1,1)$ 出发走到 $(n,m)$ ，每次只能向右（列数+1）或下（行数+1）走一个，将途径的所有格子的分数计入总分，然后再每次向左或向上走回 $(1,1)$
  - ▶ 除 $(1,1)$ 和 $(n,m)$ 外，两条路径不能相交，求最大总分
- 
- ▶ 难度1  $n,m \leq 50$
  - ▶ 难度2  $n,m \leq 500$

# NOIP2008 传纸条

- ▶ 首先，走过去走回来不相交完全可以转化为2个人同时走过去，一次走一步不相遇

# NOIP2008 传纸条

- ▶ 首先，走过去走回来不相交完全可以转化为2个人同时走过去，一次走一步不相遇
- ▶ 难度1
- ▶ 用 $f[x1][y1][x2][y2]$ 记录2个人所在的位置
- ▶  $f[x1][y1][x2][y2] \rightarrow f[x1(+1)][y1(+1)][x2(+1)][y2(+1)]$
- ▶ 由于每次同时走一步，如果线路交叉则一定会同时走到一起，排除这样的状态即可，4D0D

# NOIP2008 传纸条

- ▶ 首先，走过去走回来不相交完全可以转化为2个人同时走过去，一次走一步不相遇
- ▶ 难度1
- ▶ 用 $f[x1][y1][x2][y2]$ 记录2个人所在的位置
- ▶  $f[x1][y1][x2][y2] \rightarrow f[x1(+1)][y1(+1)][x2(+1)][y2(+1)]$
- ▶ 由于每次同时走一步，如果线路交叉则一定会同时走到一起，排除这样的状态即可，4D0D
- ▶ 难度2
- ▶ 由于同时走，因此 $x1+y1=x2+y2$ ，即 $y2=x1+y1-x2$ ，其它状态不合法
- ▶ 因此完全没必要记录 $y2$ ，3D0D

# NOIP2015 子串

有两个仅包含小写英文字母的字符串  $A$  和  $B$ 。

现在要从字符串  $A$  中取出  $k$  个互不重叠的非空子串，然后把这  $k$  个子串按照其在字符串  $A$  中出现的顺序依次连接起来得到一个新的字符串。请问有多少种方案可以使得这个新串与字符串  $B$  相等？

注意：子串取出的位置不同也认为是不同的方案。

对于第 1 组数据： $1 \leq n \leq 500, 1 \leq m \leq 50, k = 1$ ;

对于第 2 组至第 3 组数据： $1 \leq n \leq 500, 1 \leq m \leq 50, k = 2$ ;

对于第 4 组至第 5 组数据： $1 \leq n \leq 500, 1 \leq m \leq 50, k = m$ ;

对于第 1 组至第 7 组数据： $1 \leq n \leq 500, 1 \leq m \leq 50, 1 \leq k \leq m$ ;

对于第 1 组至第 9 组数据： $1 \leq n \leq 1000, 1 \leq m \leq 100, 1 \leq k \leq m$ ;

对于所有 10 组数据： $1 \leq n \leq 1000, 1 \leq m \leq 200, 1 \leq k \leq m$ 。

# NOIP2015 子串

- ▶ 一个经典的匹配DP，经典思路至少包括记录A串前i位与B串前j位相匹配
- ▶ 考虑哪些其他因素将影响决策

# NOIP2015 子串

- ▶ 一个经典的匹配DP，经典思路至少包括记录A串前i位与B串前j位相匹配
- ▶ 考虑哪些其他因素将影响决策
- ▶ 你至少需要用 $f[i][j][k][1/0]$ 记录A串前i位扣出k个子串和B串前j位匹配，且A[i]是/否包含在第k个子串中的方案数
- ▶ 然后看A[i+1]与B[j+1]

# NOIP2015 子串

- ▶ 一个经典的匹配DP，经典思路至少包括记录A串前i位与B串前j位相匹配
- ▶ 考虑哪些其他因素将影响决策
- ▶ 你至少需要用 $f[i][j][k][1/0]$ 记录A串前i位扣出k个子串和B串前j位匹配，且A[i]是/否包含在第k个子串中的方案数
- ▶ 然后看A[i+1]与B[j+1]
- ▶ 若 $A[i+1] \neq B[j+1]$ ， $f[i][j][k][0] + f[i][j][k][1] \rightarrow f[i+1][j][k][0]$
- ▶ 若 $A[i+1] = B[j+1]$ ，分以下情况：
  - ▶ 1) 抛弃A[i+1]，不加入任何子串，与不等的情况相同： $f[i][j][k][0] + f[i][j][k][1] \rightarrow f[i+1][j][k][0]$
  - ▶ 2) 将A[i+1]加入上一个子串， $f[i][j][k][1] \rightarrow f[i+1][j+1][k][1]$
  - ▶ 3) 将A[i+1]加入下一个子串， $f[i][j][k][0] + f[i][j][k][1] \rightarrow f[i+1][j+1][k+1][1]$

# NOIP2015 子串

- ▶ 时间复杂度是 $O(nmk)$ ，可以接受
- ▶ 空间复杂度是 $O(nmk)$ ，难以承受
- ▶ 但 $f[i+1][\dots]$ 只和 $f[i][\dots]$ 有关，所以滚动第一位即可

# NOIP2016 换教室

## 题目描述

展开

对于刚上大学的牛牛来说，他面临的第一个问题是如何根据实际情况申请合适的课程。

在可以选择的课程中，有  $2n$  节课程安排在  $n$  个时间段上。在第  $i$  ( $1 \leq i \leq n$ ) 个时间段上，两节内容相同的课程同时在不同的地点进行，其中，牛牛预先被安排在教室  $c_i$  上课，而另一节课程在教室  $d_i$  进行。

在不提交任何申请的情况下，学生们需要按时间段的顺序依次完成所有的  $n$  节安排好的课程。如果学生想更换第  $i$  节课程的教室，则需要提出申请。若申请通过，学生就可以在第  $i$  个时间段去教室  $d_i$  上课，否则仍然在教室  $c_i$  上课。

由于更换教室的需求太多，申请不一定能获得通过。通过计算，牛牛发现申请更换第  $i$  节课程的教室时，申请被通过的概率是一个已知的实数  $k_i$ ，并且对于不同课程的申请，被通过的概率是互相独立的。

学校规定，所有的申请只能在学期开始前一次性提交，并且每个人只能选择至多  $m$  节课程进行申请。这意味着牛牛必须一次性决定是否申请更换每节课的教室，而不能根据某些课程的申请结果来决定其他课程是否申请；牛牛可以申请自己最希望更换教室的  $m$  门课程，也可以不用完这  $m$  个申请的机会，甚至可以一门课程都不申请。

因为不同的课程可能会被安排在不同的教室进行，所以牛牛需要利用课间时间从一间教室赶到另一间教室。

牛牛所在的大学有  $v$  个教室，有  $e$  条道路。每条道路连接两间教室，并且是可以双向通行的。由于道路的长度和拥堵程度不同，通过不同的道路耗费的体力可能会有所不同。当第  $i$  ( $1 \leq i \leq n-1$ ) 节课结束后，牛牛就会从这节课的教室出发，选择一条耗费体力最少的路径前往下一节课的教室。

现在牛牛想知道，申请哪几门课程可以使他因在教室间移动耗费的体力值的总和的期望值最小，请你帮他求出这个最小值。

【提示】

1. 道路中可能会有多条双向道路连接相同的两间教室。也有可能道路两端连接的是同一间教室。
2. 请注意区分  $n, m, v, e$  的意义， $n$  不是教室的数量， $m$  不是道路的数量。

| 测试点 | $n$         | $m$         | $v$        | 特殊性质1 | 特殊性质2 |
|-----|-------------|-------------|------------|-------|-------|
| 1   | $\leq 1$    | $\leq 1$    | $\leq 300$ | ×     | ×     |
| 2   | $\leq 2$    | $\leq 0$    | $\leq 20$  |       |       |
| 3   |             | $\leq 1$    | $\leq 100$ |       |       |
| 4   |             | $\leq 2$    | $\leq 300$ |       |       |
| 5   | $\leq 3$    | $\leq 0$    | $\leq 20$  | √     | √     |
| 6   |             | $\leq 1$    | $\leq 100$ | ×     | ×     |
| 7   |             | $\leq 2$    | $\leq 300$ |       | ×     |
| 8   | $\leq 10$   | $\leq 0$    | $\leq 20$  | √     | √     |
| 9   |             | $\leq 1$    | $\leq 20$  | ×     | ×     |
| 10  |             | $\leq 2$    | $\leq 100$ |       | ×     |
| 11  |             | $\leq 10$   | $\leq 300$ |       | √     |
| 12  | $\leq 20$   | $\leq 0$    | $\leq 20$  | √     | ×     |
| 13  |             | $\leq 1$    | $\leq 100$ | ×     |       |
| 14  |             | $\leq 2$    | $\leq 300$ | √     |       |
| 15  |             | $\leq 20$   | $\leq 300$ | ×     | √     |
| 16  | $\leq 300$  | $\leq 0$    | $\leq 20$  |       | ×     |
| 17  |             | $\leq 1$    | $\leq 100$ |       |       |
| 18  |             | $\leq 2$    | $\leq 300$ | √     | √     |
| 19  | $\leq 2000$ | $\leq 300$  | $\leq 300$ | ×     | ×     |
| 20  |             | $\leq 0$    | $\leq 20$  |       |       |
| 21  |             | $\leq 1$    | $\leq 20$  |       |       |
| 22  |             | $\leq 2$    | $\leq 100$ |       |       |
| 23  |             | $\leq 2000$ | $\leq 300$ |       |       |
| 24  |             |             | $\leq 300$ |       |       |
| 25  |             |             |            |       |       |

洛谷 Luogu  
Enjoy Coding Life

特殊性质1:图上任意两点  $a_i, b_i, a_i \neq b_i$  间,存在一条耗费体力最少的路径只包含一条道路。

特殊性质2:对于所有的  $1 \leq i \leq n, k_i = 1$ 。

# NOIP2016 换教室

- ▶ 这道题目比较复杂，我们需要拆分开来思考
- ▶ 首先，这道题在DP题里夹杂了一个小图论

牛牛所在的大学有  $v$  个教室，有  $e$  条道路。每条道路连接两间教室，并且是可以双向通行的。由于道路的长度和拥堵程度不同，通过不同的道路耗费的体力可能会有所不同。当第  $i$  ( $1 \leq i \leq n - 1$ ) 节课结束后，牛牛就会从这节课的教室出发，选择一条耗费体力最少的路径前往下一节课的教室。

- ▶  $v \leq 300$ ，所以提前用一个floyd预处理任意两个教室之间的体力消耗

# NOIP2016 换教室

- ▶ 这道题目比较复杂，我们需要拆分开来思考
- ▶ 首先，这道题在DP题里夹杂了一个小图论

牛牛所在的大学有  $v$  个教室，有  $e$  条道路。每条道路连接两间教室，并且是可以双向通行的。由于道路的长度和拥堵程度不同，通过不同的道路耗费的体力可能会有所不同。当第  $i$  ( $1 \leq i \leq n - 1$ ) 节课结束后，牛牛就会从这节课的教室出发，选择一条耗费体力最少的路径前往下一节课的教室。

- ▶  $v \leq 300$ ，所以提前用一个floyd预处理任意两个教室之间的体力消耗
- ▶ 问题转化为：
- ▶ 给定一个序列 $\{p_i\}$ ，其中 $p_i = a_i$ ，允许对最多 $m$ 个位置提出申请，申请将使得 $p_i$ 被修改为 $b_i$ ，已知 $dis$ 数组，要求最小化 $\sum dis[p_i][p_{i+1}]$ 的期望值

# NOIP2016 换教室

- ▶ 普及一下期望值的概念

# NOIP2016 换教室

- ▶ 普及一下期望值的概念
- ▶ 事实上， $\text{dis}[p_i][p(i+1)]$ 这一段体力消耗，与 $p(i-1)$ 之前的和 $p(i+2)$ 之后的教室选择没有任何关系
- ▶ 但是它和 $p_i$ 是否申请更换、 $p(i+1)$ 是否申请更换都有关系
- ▶ 实际上我们考虑一下状态和决策

# NOIP2016 换教室

- ▶ 普及一下期望值的概念
- ▶ 事实上， $\text{dis}[p_i][p(i+1)]$ 这一段体力消耗，与 $p(i-1)$ 之前的和 $p(i+2)$ 之后的教室选择没有任何关系
- ▶ 但是它和 $p_i$ 是否申请更换、 $p(i+1)$ 是否申请更换都有关系
- ▶ 实际上我们考虑一下状态和决策
- ▶ 决策是某一个课程要不要申请更换教室
- ▶ 前一次是否申请更换，至今申请了多少次，都会影响决策

# NOIP2016 换教室

- ▶ 普及一下期望值的概念
- ▶ 事实上， $\text{dis}[p_i][p(i+1)]$ 这一段体力消耗，与 $p(i-1)$ 之前的和 $p(i+2)$ 之后的教室选择没有任何关系
- ▶ 但是它和 $p_i$ 是否申请更换、 $p(i+1)$ 是否申请更换都有关系
- ▶ 实际上我们考虑一下状态和决策
- ▶ 决策是某一个课程要不要申请更换教室
- ▶ 前一次是否申请更换，至今申请了多少次，都会影响决策
- ▶ 因此我们用 $f[i][j][1/0]$ 来表示，已经处理了前 $i$ 个课程，消耗掉了 $j$ 次申请机会， $p_i$ 是/否申请更换的最小期望体力消耗

# NOIP2016 换教室

- ▶ 那么状态转移方程其实很好思考

# NOIP2016 换教室

- ▶ 那么状态转移方程其实很好思考
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j][0]$
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j+1][1]$

# NOIP2016 换教室

- ▶ 那么状态转移方程其实很好思考
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j][0]$
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j+1][1]$
- ▶  $f[i][j][0] = \min\{f[i-1][j][0] + \text{dis}[a(i-1)][a_i],$   
▶  $f[i-1][j][1] + (1-k(i-1)) * \text{dis}[a(i-1)][a_i] + k(i-1) * \text{dis}[b(i-1)][a_i] \}$
- ▶  $f[i][j][1] = \min\{f[i-1][j-1][0] + (1-k_i) * \text{dis}[a(i-1)][a_i] + k_i * \text{dis}[a(i-1)][b_i],$   
▶  $f[i-1][j-1][1]$   
▶  $+ (1-k_i) * ((1-k(i-1)) * \text{dis}[a(i-1)][a_i] + k(i-1) * \text{dis}[b(i-1)][a_i])$   
▶  $+ k_i * ((1-k(i-1)) * \text{dis}[a(i-1)][b_i] + k(i-1) * \text{dis}[b(i-1)][b_i]) \}$

# NOIP2016 换教室

- ▶ 那么状态转移方程其实很好思考
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j][0]$
- ▶  $f[i][j][0/1] \rightarrow f[i+1][j+1][1]$
- ▶  $f[i][j][0] = \min\{f[i-1][j][0] + \text{dis}[a(i-1)][a_i],$   
▶  $f[i-1][j][1] + (1-k(i-1)) * \text{dis}[a(i-1)][a_i] + k(i-1) * \text{dis}[b(i-1)][a_i] \}$
- ▶  $f[i][j][1] = \min\{f[i-1][j-1][0] + (1-k_i) * \text{dis}[a(i-1)][a_i] + k_i * \text{dis}[a(i-1)][b_i],$   
▶  $f[i-1][j-1][1]$   
▶  $+ (1-k_i) * ((1-k(i-1)) * \text{dis}[a(i-1)][a_i] + k(i-1) * \text{dis}[b(i-1)][a_i])$   
▶  $+ k_i * ((1-k(i-1)) * \text{dis}[a(i-1)][b_i] + k(i-1) * \text{dis}[b(i-1)][b_i]) \}$
- ▶ 2D0D,  $O(n^2)$ 解决

# NOIP2014 飞扬的小鸟

**Flappy Bird** 是一款风靡一时的休闲手机游戏。玩家需要不断控制点击手机屏幕的频率来调节小鸟的飞行高度，让小鸟顺利通过画面右方的管道缝隙。如果小鸟一不小心碰到了水管或者掉在地上的话，便宣告失败。

为了简化问题，我们对游戏规则进行了简化和改编：

游戏界面是一个长为  $n$ ，高为  $m$  的二维平面，其中有  $k$  个管道（忽略管道的宽度）。

小鸟始终在游戏界面内移动。小鸟从游戏界面最左边任意整数高度位置出发，到达游戏界面最右边时，游戏完成。

小鸟每个单位时间沿横坐标方向右移的距离为 1，竖直移动的距离由玩家控制。如果点击屏幕，小鸟就会上升一定高度  $x$ ，每个单位时间可以点击多次，效果叠加；如果不点击屏幕，小鸟就会下降一定高度  $y$ 。小鸟位于横坐标方向不同位置时，上升的高度  $x$  和下降的高度  $y$  可能互不相同。

小鸟高度等于 0 或者小鸟碰到管道时，游戏失败。小鸟高度为  $m$  时，无法再上升。

现在，请你判断是否可以完成游戏。如果可以，输出最少点击屏幕数；否则，输出小鸟最多可以通过多少个管道缝隙。

## 【数据范围】

对于 30% 的数据：  $5 \leq n \leq 10, 5 \leq m \leq 10, k = 0$ ，保证存在一组最优解使得同一单位时间最多点击屏幕 3 次；

对于 50% 的数据：  $5 \leq n \leq 20, 5 \leq m \leq 10$ ，保证存在一组最优解使得同一单位时间最多点击屏幕 3 次；

对于 70% 的数据：  $5 \leq n \leq 1000, 5 \leq m \leq 100$ ；

对于 100% 的数据：  $5 \leq n \leq 10000, 5 \leq m \leq 1000, 0 \leq k < n, 0 < x, y < m, 0 < p < n, 0 \leq l < h \leq m, l + 1 < h$ 。

# NOIP2014 飞扬的小鸟

- ▶ 题意理解并不困难，不进行简化

# NOIP2014 飞扬的小鸟

- ▶ 题意理解并不困难，不进行简化
- ▶ 实际上，题目的阶段划分相当明显
- ▶ 很容易得出，以所在列为阶段，点击次数为目标值，那么影响后续决策的其实只有当前所处的位置

# NOIP2014 飞扬的小鸟

- ▶ 题意理解并不困难，不进行简化
- ▶ 实际上，题目的阶段划分相当明显
- ▶ 很容易得出，以所在列为阶段，点击次数为目标值，那么影响后续决策的其实只有当前所处的位置
- ▶ 一种朴素的决策分类为：在该列点击几次
- ▶  $f[i][j] + k \rightarrow f[i+1][j+x*k]$   $f[i][j] \rightarrow f[i+1][j-y]$
- ▶ 但是这样状态是 $O(nm)$ 的，转移是 $O(m)$ 的，2D1D，复杂度难以接受

# NOIP2014 飞扬的小鸟

- ▶ 实际上是思维僵化了，换一个角度思考

# NOIP2014 飞扬的小鸟

- ▶ 实际上是思维僵化了，换一个角度思考
- ▶ 单位时间点击的次数不限，这其实是一个完全背包
- ▶ 事实上，当鸟处于第 $i$ 列高度 $j$ 时，有3种决策——
  - ▶ 1. 点一下，跳到下一列；
  - ▶ 2. 点一下，在本列内继续上升；
  - ▶ 3. 不点，降落到下一列

# NOIP2014 飞扬的小鸟

- ▶ 实际上是思维僵化了，换一个角度思考
- ▶ 单位时间点击的次数不限，这其实是一个完全背包
- ▶ 事实上，当鸟处于第 $i$ 列高度 $j$ 时，有3种决策——
  - ▶ 1. 点一下，跳到下一列；
  - ▶ 2. 点一下，在本列内继续上升；
  - ▶ 3. 不点，降落到下一列
- ▶ 因此我们可以借用完全背包的思维，不去考虑已处理的 $f[i][j]$ 到底是如何更新得到的，而是通过严格把控转移的处理顺序来实现正确的转移逻辑。

# NOIP2014 飞扬的小鸟

- ▶ 我们注意到，根据题目描述：

# NOIP2014 飞扬的小鸟

- ▶ 我们注意到，根据题目描述：
- ▶ 进入后一列的方法只有跳和掉落；
- ▶ 只有进入该列时选择跳，才能在本列继续跳；
- ▶ 一旦通过降落进入本列，就不能再在本列进行操作

# NOIP2014 飞扬的小鸟

- ▶ 我们注意到，根据题目描述：
- ▶ 进入后一列的方法只有跳和掉落；
- ▶ 只有进入该列时选择跳，才能在本列继续跳；
- ▶ 一旦通过降落进入本列，就不能再在本列进行操作
- ▶ 因此处理顺序变得十分明了——
- ▶ 1. 处理从前一列跳跃进入（点击次数+1）
- ▶ 2. 从低往高处理本列内继续向上跳跃（点击次数+1）
- ▶ 3. 处理从前一列掉落进入（点击次数不变）

# NOIP2014 飞扬的小鸟

- ▶ 我们注意到，根据题目描述：
- ▶ 进入后一列的方法只有跳和掉落；
- ▶ 只有进入该列时选择跳，才能在本列继续跳；
- ▶ 一旦通过降落进入本列，就不能再在本列进行操作
- ▶ 因此处理顺序变得十分明了——
- ▶ 1. 处理从前一列跳跃进入（点击次数+1）
- ▶ 2. 从低往高处理本列内继续向上跳跃（点击次数+1）
- ▶ 3. 处理从前一列掉落进入（点击次数不变）
  
- ▶ 边界情况：遇到水管，点击次数置为+INF；向上跳跃超过m时不会死亡，仍然能更新f[i][m]

# 作业

- ▶ PPT中附有链接，去洛谷上AC较难题
- ▶ 如果还有时间可以写一写中等题
- ▶ 如果基础不太好的可以放弃难题，但要求快速刷完所有简单题，写代码与调试时间不可超过半小时